



TaskFlow

Team Task Tracker — Technical Documentation

Axionix Frontend Engineering Internship Assignment

Dependencies · Application & Data Flow · Component Inventory · Database Layout

Generated 16 September 2026

1. Overview

TaskFlow is a responsive team task tracker: users sign in, browse project dashboards, and manage tasks on a Kanban board or table view. It is a single-page React application talking to a mock REST API (json-server), with Redux Toolkit + RTK Query acting as the data-service layer between the UI and that API.

Tech stack at a glance

- React 19 (JavaScript) + Vite — component framework and dev/build tooling
- Redux Toolkit + RTK Query — global state and the data-service layer (all HTTP access goes through it)
- React Router v7 — client-side routing, protected routes, URL-synced filters
- Tailwind CSS v4 — utility-first styling
- Headless UI — accessible modal/dialog primitives (focus trap, Escape-to-close)
- json-server — mock REST API backing the app, with a custom middleware for simulated failures
- Vitest + React Testing Library + MSW — automated test suite

Folder layout

src/	
% % % api/apiSlice.js	RTK Query service layer (all HTTP access)
% % % app/store.js	Redux store
% % % features/	
% % % % auth/	LoginPage, ProtectedRoute, authSlice
% % % % dashboard/	DashboardPage
% % % % projects/	ProjectsPage, ProjectDetailsPage, CreateProjectModal
% % % % tasks/	TaskBoard (Kanban), TaskTable, TaskFormModal, filters
% % % components/	Shared UI: Button, TextField, Modal, Badge, Avatar,
% % % layouts/AppShell.jsx	ErrorState, EmptyState, Skeleton, ErrorBoundary
% % % pages/NotFoundPage.jsx	Sidebar + header + mobile bottom nav + <Outlet/>
% % % utils/date.js	isOverdue / isDueSoon / formatDate

2. Dependencies

Every package in package.json and what it is actually used for in this project.

2.1 Runtime dependencies

Package	Version	Purpose in TaskFlow
@headlessui/react	^2.2.10	Accessible Dialog used by Modal.jsx — gives every modal (task form, create-project, delete-confirm) focus trapping and Escape-to-close for free.
@reduxjs/toolkit	^2.12.0	configureStore, createSlice and RTK Query (createApi). This is the backbone of the app: authSlice for session state, apiSlice for all server data.
date-fns	^4.4.0	Date parsing/formatting and day-difference math behind isOverdue, isDueSoon and formatDate (src/utls/date.js).
lucide-react	^1.45.0	Icon set used across the UI — nav icons, form field icons, status/priority icons, buttons.
react	^19.2.8	Core UI library.
react-dom	^19.2.8	React's DOM renderer, mounts the app in main.jsx.
react-redux	^9.3.0	React bindings for the Redux store (<Provider>, useSelector, useDispatch).
react-router-dom	^7.18.3	Routing: BrowserRouter, protected routes, nested layout routes, and useSearchParams for URL-synced task filters.

2.2 Development & tooling dependencies

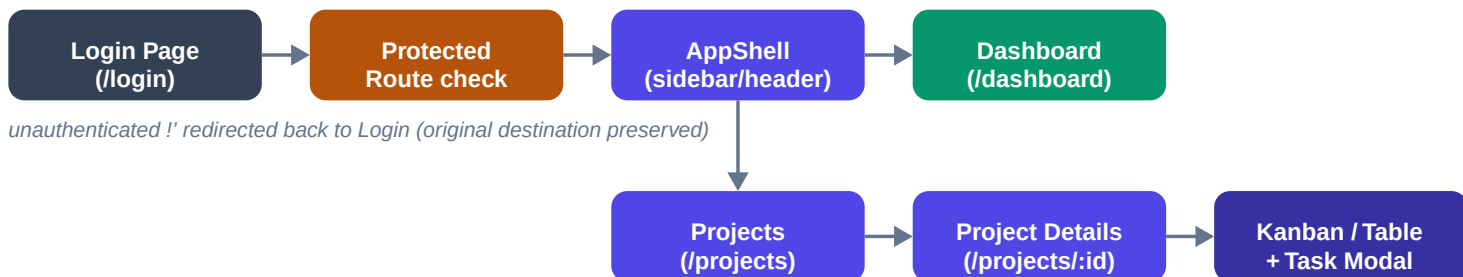
Package	Version	Purpose in TaskFlow
@tailwindcss/vite	^4.3.3	Vite plugin that wires Tailwind CSS v4 into the build — no separate PostCSS config needed.
tailwindcss	^4.3.3	Utility-first CSS framework used for all styling.
vite	^8.3.0	Dev server (HMR) and production bundler.
@vitejs/plugin-react	^6.1.1	Vite's React plugin — JSX transform and Fast Refresh.
oxlint	^1.81.0	Fast linter bundled with the Vite React template.
vitest	^5.0.0	Test runner (Vite-native, Jest-compatible API).
@testing-library/react	^16.3.3	Renders React components in tests and queries them the way a user would.
@testing-library/user-event	^14.6.7	Simulates realistic user interactions (typing, clicking, keyboard) in tests.
@testing-library/jest-dom	^7.0.1	Extra DOM matchers for Vitest, e.g. toBeInTheDocument(), toHaveFocus().
jsdom	^29.1.1	Browser-like DOM implementation Vitest runs tests against.
msw	^2.15.0	Mock Service Worker — intercepts HTTP in tests so they don't depend on json-server actually running.
json-server	^0.17.4	The mock backend itself. Pinned to 0.17 because the newer 1.0 beta dropped custom middleware support, which this project's simulated-failure feature depends on.

Package	Version	Purpose in TaskFlow
concurrently	^10.0.5	Runs the Vite dev server and json-server together with one command (npm run dev:all).
@types/react, @types/react-dom	latest	Editor type hints only (this is a JavaScript project, not TypeScript) — no effect on the build or runtime.

3. Application & Data Flow

3.1 Screen / navigation flow

How a visitor moves through the app, and how the router gates access.



unauthenticated !' redirected back to Login (original destination preserved)

any unmatched URL inside the protected area !' NotFoundPage, rendered within AppShell (not a bare page)

3.2 Data flow (RTK Query request cycle)

Every read/write of projects, tasks or users follows the same cycle — this is what "components never touch fetch or localStorage directly" means in practice:

- 1. A component calls a generated hook, e.g. `useGetTasksQuery({ projectId })` or `useCreateTaskMutation()`.
- 2. RTK Query (`src/api/apiSlice.js`) builds the HTTP request and sends it to json-server at `VITE_API_URL` (default `http://localhost:3001`).
- 3. json-server reads/writes `db.json` and returns a JSON response (or, if the request carries `?forceError=true`, the custom middleware short-circuits it with a 500).
- 4. RTK Query stores the result in its cache, tagged by type (Project/Task/User) and id, and exposes `{ data, isLoading, isError }` back to the component.
- 5. A mutation (create/update/delete) invalidates the relevant cache tags, so any query depending on that data (e.g. the Kanban board after a task edit) automatically refetches and re-renders — no manual state syncing required.

3.3 Authentication flow

- LoginPage submits `{ email, password }` to a custom login mutation (still routed through `apiSlice`).
- That mutation looks the user up via `GET /users?email=...`, then checks the fixed demo password client-side (json-server has no real auth).
- On success, `authSlice.sessionStarted` stores the user's id in Redux state and in `localStorage` (for persistence across refresh).
- ProtectedRoute reads the session from Redux on every navigation; if absent, it redirects to `/login` and remembers the originally requested URL to return to after sign-in.
- Logout clears both Redux state and `localStorage`.

4. Component Inventory

What every file in `src/` does. Grouped the same way the codebase is organized.

4.1 Entry points

File	What it does
<code>main.jsx</code>	Mounts the React root; wraps the app in <code>ErrorBoundary</code> and the Redux <code><Provider></code> .
<code>App.jsx</code>	Defines every route: public <code>/login</code> , and the protected group (<code>/dashboard</code> , <code>/projects</code> , <code>/projects/:id</code> , catch-all 404) nested inside <code>AppShell</code> .

4.2 components/ — shared UI primitives

Component	What it does
<code>Avatar.jsx</code>	Colored circular initials avatar derived from a user's name — no external image dependency.
<code>Badge.jsx</code>	<code>StatusBadge</code> and <code>PriorityBadge</code> — colored pill labels for task status/priority; text is always shown alongside color.
<code>Button.jsx</code>	Shared button: primary/secondary/danger/ghost variants, built-in loading-spinner state, disables itself while pending.
<code>EmptyState.jsx</code>	Reusable "nothing to show" placeholder — icon, message, optional action button.
<code>ErrorBoundary.jsx</code>	Class component that catches unexpected rendering errors anywhere in the tree and shows a recovery screen instead of a blank page.
<code>ErrorState.jsx</code>	Reusable "couldn't load your data" card with a Retry button; used whenever an RTK Query request fails.
<code>Modal.jsx</code>	Wraps Headless UI's <code>Dialog</code> with consistent chrome (title, close button, sizes); every modal in the app is built on this.
<code>Skeleton.jsx</code>	A pulsing gray placeholder block, composed into the per-page loading skeletons.
<code>TextField.jsx</code>	Labeled text input with optional left icon, right-side element (e.g. password show/hide), and inline validation error text.

4.3 features/auth/

File	What it does
<code>LoginPage.jsx</code>	Sign-in screen: validated email/password form, show/hide password, loading + error states, demo-credentials hint.
<code>ProtectedRoute.jsx</code>	Route guard — redirects unauthenticated visitors to <code>/login</code> and remembers where they were headed.
<code>authSlice.js</code>	Redux slice holding the signed-in user's id; persists it to <code>localStorage</code> and rehydrates on refresh.
<code>constants.js</code>	The fixed demo email/password shared between the login form logic and its on-screen hint text.

4.4 features/dashboard/

File	What it does
<code>DashboardPage.jsx</code>	Total/TODO/IN_PROGRESS/DONE counts, overdue and due-this-week task lists, an overall completion progress bar; handles loading, error and populated states.

4.5 features/projects/

File	What it does
ProjectsPage.jsx	Searchable grid of all projects, "New Project" entry point, loading/empty/error states.
ProjectCard.jsx	One project's summary: name, description, progress bar, member avatars, task count.
CreateProjectModal.jsx	Validated form to create a new project (name, description).
ProjectDetailsPage.jsx	A single project's header + progress, the Kanban/Table view toggle, task filters, and wiring for the task create/edit/delete modals — the main task-management screen.

4.6 features/tasks/

File	What it does
TaskBoard.jsx	Kanban board: three status columns (To Do / In Progress / Done), each a list of TaskCards.
TaskCard.jsx	One task on the board — title, priority badge, due date (flagged red if overdue), assignee avatar, and an accessible <select> to change status (used instead of drag-and-drop, with a keyboard-friendly alternative as the assignment FAQ allows).
TaskTable.jsx	Alternate list/table view of the same tasks — title, status, priority, due date, assignee as sortable columns.
TaskFilters.jsx	Search box plus status/priority/assignee/sort dropdowns; every change is written to the page's URL query string.
TaskFormModal.jsx	Create/edit task form — title, description, status, priority, assignee, due date — with validation and a Delete Task entry point when editing.
DeleteConfirmDialog.jsx	Confirmation modal shown before a task is permanently deleted.
filterTasks.js	Pure function that filters and sorts a task array by the current filter state; covered directly by unit tests.

4.7 layouts/, pages/, api/, app/, utils/

File	What it does
layouts/AppShell.jsx	App-wide layout: desktop sidebar nav, header with current-user avatar + logout, mobile bottom tab bar, and the <Outlet/> that renders the active page.
pages/NotFoundPage.jsx	Friendly 404 screen with a link back to the dashboard.
api/apiSlice.js	The RTK Query service layer — every HTTP call (login, users, projects, tasks CRUD) is defined here as an endpoint; this is the only file that talks to the network.
app/store.js	Redux store configuration — combines authSlice and apiSlice's reducer, adds RTK Query's middleware.
utils/date.js	isOverdue, isDueSoon and formatDate — pure date-fns-backed helpers used by the dashboard and task cards.

5. Database Layout (db.json)

The mock backend is a single db.json file served by json-server as a REST API. It has three top-level collections. Seed data: 5 users, 3 projects, 15 tasks.

5.1 Entity-relationship overview



5.2 users

Field	Type	Notes
id	string (PK)	e.g. "u1"... "u5".
name	string	Displayed in avatars, assignee lists, and the header's current-user area.
email	string	Unique; used as the sign-in identifier (looked up via GET /users?email=...).

5.3 projects

Field	Type	Notes
id	string (PK)	e.g. "p1"... "p3".
name	string	Shown on the project card and project-details header.
description	string	Short summary shown on the card and header.
memberIds	string[] (FK !' users.id)	Members of the project; drives the avatar stack and the task-assignee dropdown scoped to this project.
createdAt	ISO datetime	Set when a project is created; not currently surfaced in the UI beyond ordering.

Note: progress % shown on a project card is computed on the fly from its tasks ($\text{DONE} \div \text{total}$) — it is not a stored field.

5.4 tasks

Field	Type	Notes
id	string (PK)	e.g. "t1"... "t15".
projectId	string (FK !' projects.id)	Which project the task belongs to.
title	string	Required field; the only mandatory field enforced on the task form besides due date.
description	string	Optional free text.
status	enum	One of TODO / IN_PROGRESS / DONE.
priority	enum	One of LOW / MEDIUM / HIGH.
assigneeId	string null (FK !' users.id)	Nullable — a task can be unassigned.

Field	Type	Notes
dueDate	date (YYYY-MM-DD)	Drives overdue / due-this-week logic on the dashboard and the red "Overdue" flag on task cards.
createdAt	ISO datetime	Set on creation; used for the "sort by creation date" option.
updatedAt	ISO datetime	Refreshed on every edit or status change.

5.5 REST endpoints (served by json-server)

Method & path	Used for
GET /users	List users; also GET /users?email=... for the login lookup.
GET /users/:id	Look up a single user (e.g. the current session's user, or a task's assignee).
GET /projects	List all projects (Projects page).
GET /projects/:id	A single project's details (Project Details page).
POST /projects	Create a project (CreateProjectModal).
GET /tasks	All tasks (Dashboard) or ?projectId=... scoped to one project (Kanban/Table).
POST /tasks	Create a task (TaskFormModal).
PATCH /tasks/:id	Edit a task, or change just its status from the Kanban card.
DELETE /tasks/:id	Delete a task (after DeleteConfirmDialog is confirmed).
any request + ?forceError=true	Intercepted by server/middleware.cjs, which returns a 500 — used to demo the error state on demand.